

Transcript

Module 16: Integrating AI Models into Full Stack Applications

Video 1: Module Overview

Thus far, you have gained substantial knowledge of software engineering and machine learning modelling. In this module you will apply the final touch to the full-stack AI solutioning process by integrating machine learning models into software applications.

This process involves us three crucial steps. First, you must persist a model in long-term permanent storage. A machine learning model typically comprises the training data and the model's parameters. Second, you need to deploy the model into production so that it can score new, unseen data. Third, you must integrate the productionised model with a software application. This ensures that input data from users can be seamlessly obtained from the software application, and the scoring outcomes from the model can be channelled back to the application to automate certain business processes or decision-making.

This three-step process ensures that business value is captured from the full-stack AI solutioning project, leading to incremental revenue or greater cost savings for business. You'll learn how to deploy a machine learning model via RESTful web service endpoints and automation and orchestration tools such as MLflow. Subsequently, you'll explore how to integrate various types of software applications with the productionised model, with a primary focus on frontend software applications.

Video 2: Overview of Model Persistence and Serving

The process to create a machine learning model typically involves training and validating the model using data. The process begins with the collection and preprocessing of a high-quality training dataset. The data preprocessing step involves important tasks such as data cleaning, data transformation and data reduction. The pre-processed dataset is then used to create features through a feature engineering process. While creating features, it is important to consider model complexity, balancing a trade-off between bias and variance. The dataset consisting of the features is then used to create several machine learning models using different combinations of the features, algorithms and hyperparameters. From here on the model are evaluated using various performance metrics to identify the best model.

As part of the model development process, it is also a good practice to validate the model by splitting the dataset into training partition and testing partition. This split validation process helps to assess whether the model suffers from overfitting problem and thus ensuring its generalisability to new unseen data.

The output of this lengthy and deliberate model development process is final version of the best model, but the process of implementing a machine learning model does not stop here. In fact, the end of the model development process marks the start of another bigger process of model productionisation. The model productionisation process itself can be further divided into three steps as shown in this infographic. As you would have observed, training a machine learning model or model development is just one step in the bigger four step machine learning project lifecycle. We typically term this bigger lifecycle, the machine learning operations, or MLOps process.

As part of the model productionisation process, there are two important tasks that need to be performed. The first task is model persistence, and the second task is model serving. Model persistence refers to the process of saving a trained machine learning model so that it can be reused later without retraining from scratch. This is particularly useful when deploying models in production environments, sharing models with others, or resuming work after an interruption. There are several advantages associated with model persistence. First, model persistence, is more efficient. It saves time and computational resources by avoiding retraining each time the model needs to be used. Second, model persistence affords reproducibility and ensures that the same model can be used consistently across different sessions and applications. Third, model persistence is fundamental to model deployment. Which allows model to be integrated into real-world applications, application programming interfaces, or APIs or other software. Model persistence support version control, which helps in tracking changes and maintaining different versions of a model.

There are various methods for model persistence, depending on the nature of the machine learning model. These methods would be discussed in detail in the subsequent videos.

Model serving refers to the process of deploying a trained model into a production environment where it can receive input data and generate predictions in real-time or batch modes. It is the bridge between model training and real-world application, ensuring that users or software systems can interact with the model efficiently. There are several aspects involved in model serving. First, deployment involves hosting the model on a server or cloud platform so it can be accessed via APIs, web services, or batch processing pipelines. Second, scalability ensures the model can handle varying loads, including traffic spikes by leveraging distributed computing architecture. Third latency and performance focus on optimising inference time to provide real-time or near real-time predictions when required. Fourth, versioning and monitoring entail. keeping track of different model versions and monitoring performance for accuracy drift, latency, and failures. Finally, security and access control, including protecting the model from unauthorised access or adversarial attacks, and ensuring compliance with data privacy regulations. There are several frameworks and architecture for model serving that can be used in real-world context. We will discuss them in the subsequent videos.

Video 3: Saving and Loading scikit-learn Models

Scikit-learn is a Python library that is commonly used to train machine learning models. It is built on top of NumPy, SciPy and Matplotlib.

Scikit-learn can be used to perform both supervised learning and unsupervised learning. For supervised learning, typical classification and regression algorithms that are supported include decision tree, support vector machines, logistic regressions and random forest. For unsupervised learning, typical clustering algorithms include k-means and DBSCAN. The trained machine learning model from scikit-learn can be saved to a file and can be loaded and used later without retraining. There are two common methods for model persistence in scikit-learn. The first method uses joblib and is recommended for large models. The second method uses pickle, which is general purpose but less efficient for large models.

Let's examine the joblib approach first. Joblib is optimised for storing large Numpy array efficiently and thus is generally preferred for scikit-learn models. The basic code fragments for saving and loading a model is quite easy. Once a model has been loaded, it can be used for performing prediction on new unseen data. Let's review the advantages of using joblib. Since joblib is optimised for NumPy arrays saving models is generally faster and takes up less storage.

It can also handle large models efficiently as mentioned earlier. The syntax of joblib is also very simple.

Next, let's move on to examine pickle. Pickle is a built-in python library for serialising and de-serialising objects. For example, if you create a Python object from a class, the object can also be serialised to a file and persisted using pickle. However, since Pickle is designed for general-purpose object serialisation and de-serialisation, it is not as optimised as joblib for large numerical data. The basic code fragments for saving and loading a model is more complicated compared to joblib as they involve explicit file manipulations. But otherwise, once a model has been loaded, it can also be used for prediction using new unseen data.

Let's review the advantages of using pickle. Pickle works well with all Python objects, and not just NumPy arrays. It is also more general-purpose compared to joblib. However, the disadvantages of pickle are that it is not optimised for large NumPy array, and it is also slow and consume more memory. In summary, for persisting scikit-learn models, it is generally preferable to use joblib. In a subsequent demonstration, we will illustrate the training and persisting of a simple decision tree classifier model trained with scikit-learn using pickle. Such a model only has a few parameters and would work equally well with pickle. We will use the Iris flowers dataset, which is a classic dataset for learning machine learning. More specifically, we will train a decision tree model to classify the Iris flower species.

In this sample code, we perform a typical model training process using split validation to assess model overfitting. As you would have observed, the model has a good testing accuracy of 97% relative to the training accuracy of 96%. The model is good at classifying Iris flower species. In the next sample codes, we save the model to a file using pickle and then load the model back from the same file. After loading the model, a new observation of Iris flower with the dimensions of its flower structure, namely sepal length, sepal width, petal length and petal width, are passed into the loaded model to predict its species or class. Observe that the loaded model is able to output virginica without having to retrain the model.

Finally, we can bring out the parameters of the loaded decision tree classifier model and inspect them. Observe that it is relatively easy to match the parameters to the actual tree model shown. For example, there are 5 nodes altogether with 4 features, or independent variables, corresponding to the four dimensions of the flower structure. More importantly, observe that the threshold for the root node is petal length less than or equal to 2.45. In the case of a scikit-learn decision tree classifier model for the Iris flowers dataset, the number of parameters is not a lot. Thus, the gain in performance from using joblib is not significant. But for a larger model, perhaps a decision tree classifier with more nodes and independent variables or a neural network, you should be able to observe a significant improvement. This concludes our discussion on saving and loading scikit-learn models.

Video 4: Saving and Loading Keras Models

Whereas scikit-learn focuses on conventional machine learning models that do not involve deep learning. Tensorflow/Keras focuses on deep learning models. Keras models are thus much more complex. They are typically saved in legacy formats such as SavedModel and HDF5, or hierarchical data format, as well as the new Keras format. Let's find more about the details of saving and loading Keras models.

As you know, training a deep learning model consumes much more resources and time as compared to a conventional machine learning model.

Thus, model persistence in Keras is all the more important. In fact, model persistence in Keras has some added advantages beyond reusing a loaded model for inference. A loaded deep learning model can also be used for fine-tuning and further training or retraining. Keras provides multiple ways to save and load models, including saving the entire model, saving weights only, and saving model architecture separately. We will broadly review each of these methods.

Let's begin with saving the entire model, which is typically the recommended method. This method includes the architecture of the models, the weights and training configuration. It ensures a seamless reloading of the model without requiring additional setup. The code fragments for saving and loading an entire Keras models are generally similar to those for a scikit-learn model. The second method involves the saving of the weights only. This method is useful when you want to reinitialise a model with a new architecture but retain the previously trained weights. Let's look at some scenarios when this method might be useful. A common scenario is to modify the architecture of a deep learning model by adding new layers for transfer learning. More specifically, we might add new classification layers to a pretrained model while keeping the feature extraction layers intact. Sometimes, we might want to change the input shape such as extending a 28x28 input image model to handle 32x32 images while keeping the preceding layers similar. Other times we might modify the activation functions, or layers properties such as replacing ReLU with LeakyReLU without having to be retrain from scratch. As you can see, the second method of saving the weights only can be very useful. The code fragments to save a model's weights is generally straightforward. But a code fragment for loading a model's weights is more complicated because of the need to recreate the model first. Recreating a Keras model involve defining the architecture and compiling the model.

The third method entails saving the model architectural separately. This approach can be useful when you want to recreate the model structure without storing the large weight files. Let's explore some scenarios that might benefit from this method. Sometimes you might want to deploy a model without a training weight so that you can train the model with different user data. You might also want to share your model for collaborative research and open-source sharing without the weights. This is especially useful if the weights are trained on proprietary or sensitive data. Large organisations might also find it useful to standardise their model architecture across different machine learning engineering teams. Again, for such a scenario, the standard architecture is distributed without the weights, so that teams can train on their own datasets.

The code fragments for saving a model's architecture is generally straightforward. In this example, we save the model architecture using the JSON format. But note that it is necessary to perform an explicit file operation. As for the loading of model architecture, you need to load the JSON data before recreating the model and compiling it.

When performing model persistence in Keras, you might find the use of checkpoints to be useful for saving model progress. Saving checkpoints during training helps to resume the training process if it gets interrupted for whatever reasons. There are some best practices that you might want to keep in mind when persisting a Keras model. Whenever possible, it is good to use the model's `.save()` method to ensure that the model architecture, weight and optimiser state are saved completely. You should use checkpoints to avoid losing progress if the training is expected to take a long time. Use JSON to share model architectures when model weights are not needed. Last but not least, you should ensure compatibility by keeping track of the Keras and TensorFlow versions used during model training.

Recall that Keras provides three main formats for saving models. These are SavedModel, HDF5 and Keras format. The Keras format is the newest format that was introduced in Tensorflow 2.13.

All three formats allow you to save the model architecture, weights, and optimiser state. However, HDF5 does not allow you to save the training configuration. In terms of the format type, SavedModel uses a directory with files format whereas HDF5 and Keras use a single binary file. On this note, HDF5 and Keras use of a single binary file is generally more convenient. In terms of performance, SavedModel and Keras formats are faster than HDF5. Finally, the file size of a Keras format model is the smallest and SavedModel has the largest file size. In summary, if you need to optimise for fast loading time, the newest Keras format is the most preferable one to use.

In the subsequent demonstration, we will illustrate the training and persisting of a simple convolutional neural network or CNN model for classifying shapes using Keras. We will omit the discussion on image pre-processing for simplicity. The first sample code shows the definition of the model architecture together with the training process. The next sample code shows the model saving and loading process using the Keras format. After the model has been loaded, it can be used to classify the shape in a new unseen image file. Note that in this CNN model, it can classify the four shapes of circle, square, star and triangle. This sample code shows the summary of the CNN model. Observe, that the total number of parameters in the model is almost 2 million, and the number of trainable parameters is 680,000. This number is certainly much larger than the number of parameters in our scikit-learn decision tree classifier model. Note that this model is merely used to classify four shapes. More complex deep learning models might have much greater number of parameters.

If you are adventurous enough, you can also drill down into the number of trainable parameters for each layer together with their associated training weights. This concludes our discussion on saving and loading Keras models.

Video 5: Serving Models via RESTful API Endpoints Using Python With Flask and Connexion

Recall from an earlier video that model serving refers to the process of deploying a trained model into production environment where it can receive input data and generate predictions. There are several approaches for serving a model, involving a mix of frameworks and architectures.

In terms of frameworks, there are generic frameworks that can be used to implement application programming interface or API endpoints using RESTful web services. In the Python ecosystem, these frameworks include Django and Flask with connexion. Adopting an API endpoint approach provides you with the most flexible option for customising the serving of your machine learning models and integrating them tightly into your business processes and software applications. This is a viable option to consider from the simplest scikit-learn models to the most complex business requirements involving deep learning models. However, adopting an API endpoint approach requires you to implement the coding required for the endpoints.

Different deep learning frameworks also provide their own model serving cloud services. This allows you to easily deploy your models without having to write much code. For example, Tensor Flow serving is a specialised framework for serving Tensor flow models and offers high performance inference. TorchServe is an equivalent framework designed for PyTorch models. However, these vendor specific frameworks are less flexible and has an undesirable lock-in effect. If you wish to change a model service strategy in the future, considerable effort and cost might be involved.

Of course, there are more generic MLOps automation and orchestration servers, such as MLflow models that you can use to serve your machine learning models. We will discuss more about this approach in a future video.

For now, let's focus on serving models via RESTful API endpoints with Flask and Connexion. Recall that RESTful endpoints allow user to pass input data as arguments and obtain processed results as JSON formatted data. In most cases, the input data representing a new unseen observation that go into a machine learning model consists of structured data such as numbers, e.g., age and weight and categorical labels, e.g., gender and credit rating. If the number of variables is not a lot, this input data can be passed into a RESTful web service endpoint as either path parameters or query string parameters using any HTTP verbs or methods. But as the number of independent variables of the model increases the amount of input data will also increase proportionately. In this case, it will be easier to pass in the input data as JSON formatted values in the request body of either the PUT or POST methods.

In this sample code, all three different approaches of path parameters, query string parameters and request body JSON formatted values are demonstrated. What happens if you need to upload an image to a deep learning model? Well, Flask and Connexion do allow you to perform file upload quite easily. This sample code shows you a post endpoint that allows user to upload a file such as an image. Within an endpoint handler method, you just need to load the required model from persistence storage and pass the input data to get a prediction. The prediction result can be easily returned to the user as a string value. In this demonstration, we load a scikit-learn clustering model using joblib. The input data is a simple numerical value that is passed into the endpoint as a query string parameter. The cluster label is then returned to the caller as a simple string value.

Observe that serving machine learning models via RESTful web service API endpoints is relatively straightforward. In the subsequent videos, we will discuss more about how to consume models that are served via API endpoints using both frontend and backend software applications.

Video 6: Consuming RESTful API Endpoints from a React Web Application

In earlier modules of this course, you have learnt frontend software engineering using various frameworks such as React, Angular, and Vue. These frameworks are based on client-side rendering, in which the HTML codes for rendering web pages is generated within a web browser. You will also recall that for client-side rendering web application to access data in a database to generate dynamic content, it is necessary to go through backend API endpoints created using RESTful web services. In a similar manner, a client-side rendering web application running within a web browser can leverage on machine learning models served via API endpoints to automate the delivery of business values to customers. For example, machine learning models that can predict customer preferences, can be used to provide product or service recommendations, or personalised marketing messages. This helps enhance the shopping experience and drive incremental revenue.

In this module, we'll broadly review a set of techniques for a React application to consume API endpoints that are created with RESTful web services. Some of these techniques are also applicable to other frameworks. The first technique uses, the native fetch API, which is the `fetch()` function. The `fetch()` function is built into JavaScript for making network requests over the HTTP protocol, it replaces the older `XMLHttpRequest` object and provides a cleaner, more flexible way to handle network requests. The `fetch()` function is promise based and returns a promise object, making it easier to handle asynchronous operations. It is also more readable and flexible. Supporting `async/await`, as well as the chaining of the `.then()` function for handling responses. This code fragment shows the basic syntax for the `fetch()` function.

The `fetch()` function is typically used within a React `useEffect` hook codes.

In this code fragment, the `fetch()` function is used to retrieve some data from an API endpoint and the data is used to update HTML unordered list. In the case of invoking an API endpoint to obtain a machine learning model prediction. The prediction can be used to customise the actual dynamic content that is rendered in the webpage. When working with the `fetch()` function, you can use the `async/await` keywords instead of the `.then()` function if desirable.

Beyond the `fetch()` function, `Axios` can also be used. `Axios` is a third party library that is designed to simplify API calls. This code fragment demonstrates the use of `Axios` to fetch data inside a `React` `useEffect` hook. `Axios` also provides built-in error handling, making it easier to handle API failures as compared to the `fetch()` function. For example, unlike the `fetch()` function, which does not reject on HTTP errors, for example 404 Not Found or 500 Internal Server Error, `Axios` automatically rejects the promise for non-2xx status codes. In contrast with the `fetch()` function, you will need to manually check for `response.ok` and `response.status`. The code fragments shown here illustrate the difference in error handling between the `fetch()` function and `Axios`.

Last but not least, you can also use `React Query` to invoke an API endpoint. `React Query` is a powerful library for fetching data and managing state in react applications. Some of the advantages of `React Query` include caching of data, automatic refetching of data. Support for pagination and infinite scrolling and background updates among others. This concludes our discussion on interacting with a machine learning model served via an API endpoint for a `React` web application.

Video 7: Consuming RESTful API Endpoints from an Express.js Backend

Other than the frontend web application or mobile application, it may sometimes be necessary to invoke machine learning models for making predictions on the backend of a software application.

There are two main scenarios in which this approach could be useful. First. The web application is developed using a server-side rendering technology stack, in which the code for generating the HTML is run on the web server. Second, the API endpoint of another software system, including the possibility of an external system outside of your organisation's boundary, might need to utilise your machine learning models. Recall that in earlier modules of this course. you have learnt how to develop the backend of a software system using `JavaScript` with the `Express.js` framework and `Python` using the `Flask` and `Connexion` frameworks.

In this video, we'll focus our discussion on consuming an API endpoint to interact with a machine learning model from `Express.js` with `JavaScript`. In the next video, we'll turn our attention to the `Flask` and `Connexion` frameworks with `Python`. Since we are working with `JavaScript` in `Express.js`, the techniques for consuming API Endpoints created with RESTful web services are generally similar to those of a `React` web application.

There are three HTTP clients that can be used for this purpose. First, the built-in `Fetch` API in `Node.js` from version 18 onwards can be used to invoke API endpoints. Second, the third-party library `Axios` can also be used. Last but not least, for older `Node.js` version, you can use the `node-fetch` library. The built-in `fetch()` function in `Node.js` works in a similar way, as that of a regular `JavaScript` browser environment. You can also use the `async/await` keywords to handle asynchronous processing. Likewise, the `Axios` library operates similarly in the `Node.js` environment as a regular browser environment. `Axios` also offers the same benefit over the `fetch()` within `Node.js`. If you are working with an older version of `Node.js`. That is a version before version 18. You can use the `node-fetch` library. The `fetch()` function in this library works in a similar manner as the `fetch()` function in the newer built-in `Fetch` API.

The main noticeable differences are that you'll need to explicitly install a node-fetch library with the node package manager or npm and then import the fetch() function from node-fetch. One nice feature about invoking API endpoints in Express.js is the ability to optimise API calls with caching. This helps to prevent excessive API calls. This can be achieved using the node-cache library, which is an in-memory caching library for Node.js. You can also consider using Redis. Or Remote Dictionary server, which is an open source in-memory data store that can be used as a database cache and message broker. This code fragment illustrates the use of the node-cache library to cache an API call in conjunction with the use of the Axios library, observe that if the required data is available in the cache, there's no need to invoke the same API endpoint again. Otherwise, the endpoint is invoked, and the initial data is added into the cache for reuse in the future.

This concludes our discussion on interacting with a machine learning model served via an API endpoint from a Express.js backend.

Video 8: Consuming RESTful API Endpoints from a Flask Backend

In the previous video, we have discussed how to invoke API endpoints from the backend of a software system using JavaScript with the Express.js framework. In this video, we'll focus our discussion on using Python with the Flask and Connexion frameworks.

Even though Flask and Connexion frameworks are Python based, it is not possible to use machine learning models directly unless you have access to the model architecture and the associated parameter weights. In situations in which the machine learning models are external to your software system, you will likely need to interact with the machine learning models via API endpoints. Connexion is an extension for Flask that helps with API endpoints using the OpenAPI or Swagger specifications. It provides built-in validation and allows us to define API endpoints in a structured way. While Connexion is used for defining and serving API endpoints consuming external API is typically done using requests, httpx, or other HTTP client libraries.

Let's begin with the request library, which is the most common way to consume external API endpoints in Python by making HTTP requests. The request library can be installed with the pip package manager. Recall that the four HTTP methods that are most commonly used to define API endpoints are GET for retrieving data, PUT for creating new data, POST for updating existing data and DELETE for deleting existing data. In the request library, the request.get() function is used to send a get request. If there's a need to pass any data to the GET endpoint, you can use the params parameter to pass in a Python dictionary of key-value pairs; or use one or more named arguments, for example, to get a specific record by its record identifier you can use get(url, params={"id":123}) or use get(url, id=123). The .json() function is used to parse the JSON response and the .status_code property is used to check the HTTP status code. This code fragment illustrates the use of the requests.get(url).

The requests.put function is used to send data in JSON format to a PUT endpoint. Recall that JSON formatted data is structurally similar to the dictionary data structure in Python. So, working with JSON in Python is relatively easy. For a put endpoint. The HTTP status code 201 is typically used to indicate successful creation of the new data records. You can also send data as data=data named argument, a instead of json=data, but it will be sent as form-encoded instead. This code fragment illustrates the use of the requests.put() function. Observe the use of a Python dictionary data structure to handle JSON data.

In a similar manner. The `requests.post()` function is used to send a update request to a post endpoint. For a POST endpoint the HTTP status code 200 is typically used to indicate successful updating of an existing data records.

This code fragment illustrates the use of the `requests.put()` function. Also, that it is very similar to the one for the `request.put()` function. Take note that only PUT and POST endpoints allow the sending of JSON formatted data via the body of the HTTP request.

Finally, the `requests.delete` function is used to send a DELETE request to remove an existing data record. Most delete endpoints return an empty response with the HTTP status code 200 OK or 204 no content. More often than not, you need to send the record identifier of the record to be deleted. This can be done by passing one or more named arguments to the delete function. For example, `delete(id=123)`. This code fragment illustrates the use of the request delete function. This concludes our discussion on interacting with a machine learning model served via an API endpoint from a Flask and Connexion backend.

Video 9: Serving a Model Using MLflow

Thus far, we have focused on serving machine learning models via API endpoints created using RESTful web services. These API endpoints can be consumed in the frontend and backend of software systems to integrate the predictions made by machine learning models to automate business processes and decision-making. Ultimately, consumers benefit from a better user experience and more personalised products and services. Besides using API endpoints to serve machine learning models, it is also possible to use platforms such as ML Flow to manage the entire machine learning lifecycle. More specifically, MLflow is an open-source platform designed to handle machine learning, experimentation, reproducibility, deployment, and model management. It helps AI/ML engineers to track their experiments, package models and deploy them in a scalable way.

MLflow consists of four primary components. MLflow Tracking logs and tracks experiment, parameters, metrics and artifacts. MLflow Projects standardises ML codes to ensure reproducibility. MLflow Models, facilitate model packaging, versioning, and deployment. MLflow Registry provides modern version control annotations and lifecycle management.

So, how does MLflow manages model deployment? Well, MLflow provides several deployment mechanisms, making it easier to transition models from experimentation to production. First, let's examine model logging and registration. Before deployment, MLflow helps log and register trained models. This ensures that the exact parameters, hyperparameters and environment dependencies are tracked. The model version is maintained for easier rollback and comparison. The model artifacts, including the training data are stored in a central repository. This code fragment illustrate the model logging and registration process. Observe that after logging the model can be registered in the MLflow model registry for version control.

Next, let's move on to examining model versioning and governance. ML Flows Model Registry helps to manage different version of models, enabling approval workflows for model promotion from staging to production. It also supports auditability with metadata like who trained the model and when. Finally, annotations and description help to provide insights on changes between versions. This code fragment demonstrates the process of updating a model, e.g., move from staging to production.

Third, in model packaging for deployment, MLflow models are saved in a standard format which is the ML model format. This facilitates easy loading and deployment of models.

The ML model format supports multiple types of models such as scikit-learn, PyTorch, TensorFlow and XGBoost. Containerisation can also be used for deployment in cloud or local environments. This code fragment demonstrates the loading of a model stored in the ML model format. Fourth and lastly, in terms of deployment strategies, MLflow provides multiple deployment options. Local deployment entails the running of models locally using `mlflow.pyfunc.serve` for testing. The REST API deployment strategy exposes model as REST API using MLflow's built-in server. For cloud deployment, MLflow models can be deployed on major cloud service providers such as AWS SageMaker, Azure ML or Google Cloud AI platform. A containerisation deployment strategy. Kubernetes is supported. ML can be used to deploy models as Kubernetes pods with Kserve. For maximum flexibility and customisability. It is also possible to integrate MLflow with your own MLOps pipelines. MLflow can integrate readily with CI/CD tools such as GitHub Actions, Jenkins, and Apache Airflow for automated model deployment and monitoring.

Let's review the key benefits of using MLflow for deployment. MLflow provides reproducibility and ensures that models can be recreated with the same parameters and environment. MLflow supports version control, which enables tracking of different iterations of model and allows easy rollbacks. MLflow is highly scalable and can deploy models across various cloud services and on-premises infrastructure. MLflow can be fully integrated with automation and CI/CD pipelines to automate model promotion and monitoring. In terms of monitoring and governance, MLflow keep track of deployed models, their performance and updates. As you can see, using MLflow offers much more benefits compared to simply serving your machine learning models via your API endpoints. I will strongly encourage you to try out MLflow on your own and experience the various industry best practices that we have just discussed.

Let's take a look at two demonstrations on working with MLflow. The first demonstration involves a scikit-learn decision tree classifier model. Second demonstration involves a Keras convolutional neural network for classifying shapes in images. At this juncture, MLflow is running in an Ubuntu operating system environment and there's no experiment and model registered on the server. After running the sample code to train the Scikit-learn and Keras Models, you can now see the two experiments and their model being registered on MLflow. Let's zoom in on the scikit-learn model. In the screenshots shown, you can see the model being tracked on MLflow together with the parameters used to train the model. MLflow also simplifies the tracking of the model performance metrics to help you identify the best version of your model. Finally, MLflow stores the entire artifact that represents the model ready for productionisation.

Observe that even the training data is stored in MLflow using JSON file format. This allows full reproducibility of the models outputs. From here on, we can easily load the models back from MLflow and use the loaded models to perform prediction on new unseen data. Observe that from the loaded model, you will be able to view all the trainable parameters of the model. This is similar to what you can do if you were to persist and load the model manually using joblib or pickle as discussed in an earlier video. In conclusion, MLflow simplifies and standardises machine learning model deployment across different platforms. By using model tracking, versioning and deployment tools, it enhances the reliability, scalability and maintainability of machine learning workflows. Whether deploying locally, as a REST API in the clouds or in Kubernetes MLflow provides a unified approach to managing the entire machine learning life cycle.

Video 10: Module Summary

Congratulations on completing this challenging module on managing the machine learning model lifecycle.

In real-world machine learning projects, we certainly do not stop after the model training process has been completed. Indeed as you would have learnt from the videos in this module, there are many important steps that you would need to take to productionise a machine learning model after training has been completed. Through the use of appropriate techniques, tools and best practices, in particular the MLflow platform, the efficacy and efficiency of the model productionisation process can be significantly enhanced. From this point onwards I will also encourage you to explore more about model monitoring and feedback loops to ensure that a model continues to work well post-production. In particular, model monitoring can help to identify and mitigate problems such as data drift and concept drift early.

Good luck with your learning journey and have fun.

THE END